

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

```
file = '/content/drive/MyDrive/B.TECH/5th Sem/MDM Dataset/wireless_signal_quality_dataset.xlsx - Sheet
```

```
file = '/content/wireless_signal_quality_dataset.xlsx - Sheet1.csv'
data = pd.read_csv(file)
print("first five Records")
print(data.head())
print("\nDataset Shape:", data.shape)
```

first five Records

	Signal_Strength_dBm	SNR_dB	Interference_dB	Distance_m	Frequency_GHz	\
0	-41	6	0	257	5.0	
1	-86	25	20	134	2.4	
2	-73	27	0	385	2.4	
3	-57	5	0	255	5.0	
4	-87	15	24	61	5.0	

	Channel_Bandwidth_MHz	Noise_Floor_dBm	Connected_Devices	\
0	160	-100	24	
1	160	-96	41	
2	20	-72	13	
3	160	-86	9	
4	20	-79	32	

	Estimated_Throughput_Mbps	Quality
0	18	Poor
1	55	Poor
2	81	Fair
3	15	Poor
4	21	Poor

Dataset Shape: (100, 10)

```
data.isnull().sum()
```

	0
Signal_Strength_dBm	0
SNR_dB	0
Interference_dB	0
Distance_m	0
Frequency_GHz	0
Channel_Bandwidth_MHz	0
Noise_Floor_dBm	0
Connected_Devices	0
Estimated_Throughput_Mbps	0
Quality	0

dtype: int64

```
X = data.drop("Quality", axis=1)
y = data["Quality"]
```

```
encoder = LabelEncoder()
y = encoder.fit_transform(y)
print("\nDataset Labels:")
```

Dataset Labels:

Start coding or [generate](#) with AI.

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100, max_depth=None, random_state=42)
rf.fit(x_train, y_train)
```

▼ RandomForestClassifier ⓘ ?

```
RandomForestClassifier(random_state=42)
```

```
y_pread = rf.predict(x_test)
```

```
accuracy = accuracy_score(y_test, y_pread)
print("\nAccuracy = {:.2f}%".format(accuracy*100))
```

Accuracy = 75.00%

```
accuracy = accuracy_score(y_test, y_pread)
print("\nAccuracy = {:.2f}%".format(accuracy*100))
```

Accuracy = 75.00%

```
cn = confusion_matrix(y_test, y_pread)
print("\nConfusion Matrix:")
print(cn)
```

Confusion Matrix:

```
[[ 0  1  0  0]
 [ 0  4  0  2]
 [ 0  2  1  0]
 [ 0  0  0 10]]
```

```
importance = pd.Series(
    rf.feature_importances_,
    index=X.columns
)
importance = importance.sort_values(ascending=False)
print("\nFeature Importance:")
print(importance)
```

Feature Importance:

Signal_Strength_dBm	0.395390
SNR_dB	0.178274
Estimated_Throughput_Mbps	0.164952
Interference_dB	0.073897
Noise_Floor_dBm	0.054547
Distance_m	0.052557
Connected_Devices	0.047600
Channel_Bandwidth_MHz	0.019887
Frequency_GHz	0.012896

dtype: float64

Start coding or [generate](#) with AI.